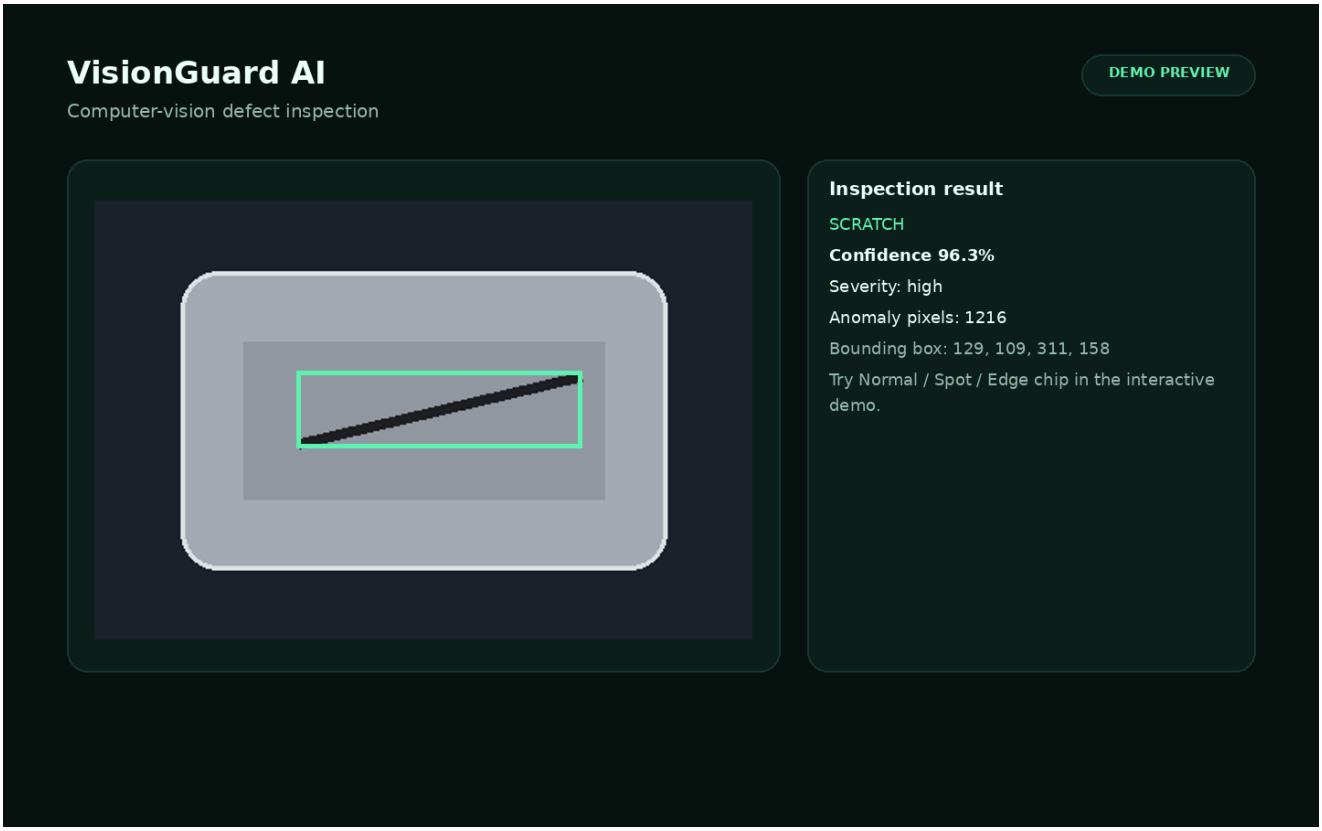
VisionGuard AI

Visual Defect Inspection Platform

Python • FastAPI • Computer Vision • Image Processing • PyTorch/YOLO-ready • Docker



Representative demo preview. Run the included application for the interactive version.

1. Problem and Solution

Manual visual inspection is slow and inconsistent. VisionGuard demonstrates an inference service that identifies surface anomalies, localizes them, scores confidence/severity and exposes the result through a browser dashboard.

Why this is strong on an AI/ML resume

The project is built as a small product rather than only a notebook. It includes a backend API, interactive frontend, repeatable local data, automated tests, Docker packaging, CI configuration, demo instructions, and clear model/engineering boundaries.

Core components

Component	Responsibility
Image Ingestion	Validates uploads and accepts repeatable built-in samples.
Preprocessing	Normalizes the image and focuses analysis on an inspection region.
Detector	The default lightweight anomaly detector makes the demo run without a GPU; model adapters allow trained detectors later.
Localization + UI	Returns a bounding box, class, confidence and severity and draws the result for the operator.

2. Architecture and Data Flow

The demo detector uses an ROI anomaly mask. The API boundary is model-agnostic, so a trained YOLO detector can replace it via ml/yolo_adapter.py.

Engineering decisions

- Keep the core demo fully local so it is reliable during interviews.
- Separate model logic from API/UI so algorithms can be upgraded without rewriting the product surface.
- Use explicit validation and automated tests so the demo can be explained as an engineering system, not only an ML experiment.
- Package with Docker and CI configuration to demonstrate repeatability.

What “production-style” means here

The repositories demonstrate production patterns, but they are portfolio systems, not claims of enterprise production deployment. A real deployment would add identity/access management, secret management, persistent observability, load testing, threat modeling, data governance, backup/rollback processes and infrastructure-specific scaling.

3. Live Demo Walkthrough

1. Start with the Scratch example that loads automatically.

2. Point out the localized bounding box and confidence score.

3. Switch to Normal, Surface Spot and Edge Chip.

4. Upload a custom image and show the same API path.

5. Explain the trained-model upgrade path with PyTorch/YOLO, versioned weights and GPU inference.

What to say while demoing

> *VisionGuard AI is a visual-quality inspection platform. An operator uploads an image or camera frame. The inference service preprocesses the image, identifies anomalous pixels inside the inspection region, classifies the defect, calculates a bounding box and severity, and returns the result to a live dashboard. I kept the default model lightweight so the portfolio demo runs on a normal laptop, while the repository has PyTorch and YOLO adapters for a trained industrial deployment. Demo order: Scratch → Normal → Spot → explain API/model swap → production improvements.*

Demo failure fallback

Keep `demo_output.json` and the preview image in the repository. If a live environment fails, walk through the saved output, explain the expected API call, then show the unit tests. This demonstrates preparedness without pretending the failed live run succeeded.

4. Resume Bullets

- Built a computer-vision inspection service that detects surface defects, localizes anomalous regions with bounding boxes, and returns confidence/severity scores through a FastAPI inference API.
- Implemented image preprocessing, region-of-interest analysis, real-time browser visualization, sample inspection workflows, request validation, Docker deployment and automated tests.
- Added pluggable PyTorch/YOLO model adapters so the lightweight local demo can be upgraded to trained deep-learning detection models without changing the API contract.

Important claim rule

Only claim technologies you can run and explain. Optional modules such as YOLO, PyTorch two-tower models, FAISS, MLflow or Airflow are included as upgrade paths where noted; do not say you trained/deployed them until you actually execute that path and record evidence/metrics.

Metrics to discuss

Precision/Recall/F1	mAP50 / mAP50-95	IoU localization	False reject rate	Inference latency

5. Interview Questions

Which metrics would you report?

Precision, recall, F1, per-class confusion matrix, mAP50/mAP50-95, localization IoU, inference latency and false-reject rate.

How do you handle class imbalance?

Targeted augmentation, weighted loss/focal loss, balanced sampling and threshold tuning per defect class.

Questions you should be ready for

- Why did you choose this algorithm instead of an alternative?
- What is the biggest failure mode in the current implementation?
- How would the architecture change at 100x traffic/data size?
- Which offline and online metrics would you monitor?
- What did you personally implement, test and debug?

6. Upgrade Roadmap

- Add authentication and role-based access.
- Persist model/retrieval artifacts with explicit versioning.
- Add structured observability: request logs, latency, error rates and model metrics.
- Create a small evaluation suite with expected outputs and regression thresholds.
- Deploy to a cloud service and record the public demo URL in the README.
- Add one real dataset experiment and document the training/evaluation results.

Final presentation advice

Do not memorize only the feature list. Be able to draw the data flow, explain one algorithm choice, show one failure case, quote one meaningful metric and describe one scalability improvement. That is what turns the project from a resume line into an interview asset.